

Volume 2

From Spreadsheet to Linked Data

Step-by-Step RDF Transformation Using OpenRefine

This volume follows:

Volume 1 — From Source Text to Structured Dataset (PDF Parsing and Dataset Construction)

This volume precedes:

Volume 3 — The Mapping Workbook

Volume 4 — GitHub Pages and Ontology Site Maintenance

Iroko Historical Society | irokosociety.org | March 2026

Introduction

This manual documents the complete workflow for transforming tabular ethnobotanical data into RDF/Turtle format using OpenRefine and the RDF Transform extension. It was developed during the conversion of the Verger Ewé Database into Linked Data using the Iroko Framework as the controlled vocabulary backbone. It covers the full dataset column structure including the Haiti, Vodou, Brazil, and Candomblé name columns introduced in the v2 schema.

The manual is written for Iroko staff and interns with no prior coding experience. Familiarity with the Iroko Framework vocabulary and a basic understanding of what RDF triples are (subject, predicate, object) will be helpful but is not required. Concepts are introduced as they become relevant.

The workflow has nine parts:

- **Part 1:** Source file preparation and encoding
- **Part 2:** OpenRefine project creation and initial data inspection
- **Part 3:** Data cleaning and controlled vocabulary standardization using GREL

- **Part 4:** URI column construction
- **Part 5:** Taxonomic reconciliation using Global Names Verifier
- **Part 6:** Namespace resolution infrastructure
- **Part 7:** RDF Transform skeleton configuration and export
- **Part 8:** Export and post-export validation
- **Part 9:** OpenRefine project preservation

A Transformation QC Checklist is provided as Appendix B. It must be completed before any Turtle file is considered ready for publication or submission. The dataset QC Checklist (which precedes this workflow) is in Volume 1, Appendix B.

NOTE: *Do not begin this workflow until the Volume 1 Production QC Checklist has been completed and signed off by the Executive Director. The transformation pipeline amplifies any errors present in the source data. Fixing a controlled vocabulary mismatch in the spreadsheet takes two minutes. Fixing it after export and reloading the skeleton takes twenty.*

Prerequisites

Software required

- **OpenRefine 3.9** — download from openrefine.org/download.html
- **RDF Transform extension** — download from github.com/AtesComp/rdf-transform. Extract and copy the folder into OpenRefine > webapp > extensions. Restart OpenRefine and confirm "RDF Transform" appears in the top menu bar.
- **Microsoft Excel or Google Sheets** — for initial file preparation
- **A plain text editor** — for verifying encoding and post-export validation. Do not use Microsoft Word for inspecting .ttl files.

NOTE: *The RDF Transform extension is version-locked to OpenRefine 3.9. Do not use a newer or older version of OpenRefine without first verifying the extension is compatible. The extension does not auto-update and may silently fail on mismatched versions.*

Vocabulary reference

Before beginning, confirm you have access to the current Iroko Framework HTML browse pages at ontology.irokosociety.org. You will need the exact concept slugs for all controlled vocabulary values in your dataset. For the Verger dataset, the three concept groups used are:

- `iroko:accessLevel` concepts — browse at </vocab/iroko-core.html>

- `iroko:medicinalUse` concepts — browse at </vocab/iroko-ewe.html>
- `iroko:ritualUse` concepts — browse at </vocab/iroko-ewe.html>

All Iroko concept URIs follow the unified flat namespace:

```
https://ontology.irokosociety.org/iroko#[concept-slug]

# Example:
https://ontology.irokosociety.org/iroko#ritual-healing-restoration
```

The complete mapping from human-readable labels to concept slugs for the Verger dataset is documented in the GREL Quick Reference in Appendix A and in the mapping workbook (Volume 3).

Confirm the Volume 1 QC checklist is complete

Open the signed-off QC checklist for the dataset version you are transforming. Verify it is dated, signed, and has no Fail marks. If any checks are marked Fail or the checklist has not been completed, stop and return to Volume 1 before proceeding.

Part 1: Source File Preparation

Why this step matters

OpenRefine cannot apply delimiter settings to Excel binary files (.xlsx). When an Excel file is imported directly, any pipe-delimited content inside cells will appear as unsplit literal text. Additionally, Excel's default CSV export uses Windows-1252 encoding, which silently drops all diacritical marks — tone marks in Yoruba, accented characters in Spanish, Portuguese, and Haitian Creole, and any Unicode character outside the basic Latin set.

Both problems must be solved before OpenRefine sees the file.

Step 1.1 — Save as CSV UTF-8 from Excel

1. Open your dataset file in Excel.
2. Go to File > Save As.
3. Select **"CSV UTF-8 (Comma delimited)"** from the format dropdown. Do not select the plain "CSV (Comma delimited)" option — that uses Windows-1252 encoding and will corrupt all diacriticals.
4. Save the file.

NOTE: In some Excel versions the UTF-8 option is labeled "CSV UTF-8 (with BOM)" instead. Both are correct. The BOM (Byte Order Mark) is a harmless header that signals UTF-8 encoding and OpenRefine handles it correctly.

Step 1.2 — Alternative: Google Sheets as a conversion bridge

If the UTF-8 CSV option is not available in your version of Excel, or if diacriticals are still being lost after export:

5. Upload the .xlsx file to Google Drive.
6. Open it with Google Sheets (it will preserve all Unicode characters).
7. Go to File > Download > Comma-separated values (.csv). Google Sheets exports UTF-8 by default.
8. Use this CSV as your OpenRefine source file.

Step 1.3 — Verify before proceeding

Open the exported CSV in a plain text editor (not Excel) and spot-check cells containing Yoruba, Spanish, Portuguese, and Haitian Creole characters. Confirm that tone marks and accents appear as actual characters rather than question marks or garbled sequences. Pay special attention to:

- Yoruba: ẹ, ọ, ẹ and tone marks on vowels
- Spanish/Portuguese: á, é, í, ó, ú, ã, ç, ñ
- Haitian Creole: è, ou, an combinations

If any of these appear garbled, re-export using the Google Sheets method before proceeding.

Part 2: OpenRefine Project Creation

Step 2.1 — Create the project

9. Open OpenRefine in your browser (it runs locally at <http://127.0.0.1:3333> by default).
10. Click "Create Project" > "This Computer" and upload your UTF-8 CSV file.
11. In the import preview screen, confirm the following settings: Parse data as **CSV / TSV / separator-based files**; Columns separated by **commas**; Character encoding **UTF-8**; Parse next 1 line(s) as column headers **checked**.
12. Give the project a descriptive name including the dataset name and date (e.g., "Verger-Ewe-RDF-2026").
13. Click "Create Project" in the upper right.

NOTE: Do not set the column separator to pipe (|) at the import stage. Your dataset uses commas between columns. Pipes appear within individual cells as multi-value separators within a single field — those are handled in Part 3 with OpenRefine's cell-splitting function.

Step 2.2 — Initial inspection

Once the project loads, scroll through the data and verify:

- All columns have loaded with correct headers.
- The column count matches the expected dataset schema (19 columns as of v2 schema).
- Diacritical marks appear correctly (Yoruba tone marks, Spanish and Portuguese accents, Haitian Creole characters).
- Pipe characters appear within cells in multi-valued fields (Yoruba Name(s), Plant Synonyms, Name (Cuba), etc.) rather than being split across multiple columns.

If diacriticals appear as garbled characters at this stage, close the project, return to Part 1, and re-export the CSV with correct encoding before proceeding.

Part 3: Data Cleaning and Controlled Vocabulary Standardization

Overview

This phase has two major components: (1) splitting multi-valued cells so each row represents a single value for reconciliation and RDF output purposes, and (2) standardizing the three controlled vocabulary columns so their values match the exact concept slugs defined in the Iroko Framework. The language-tagged name columns (Yoruba, English, Cuba, Haiti, Brazil, Lucumi, Vodou, Candomblé, Other Vernacular Names) do not require controlled vocabulary standardization — they pass through as literal strings.

Step 3.1 — Text faceting for audit

Before writing any GREL expressions, use text facets to inspect the exact values present in each controlled vocabulary column. GREL string matching is exact and case-sensitive. A single extra space, a capitalization difference, or a punctuation variant will cause a value to fall through to the fallback and not map to a concept slug.

14. Click the dropdown on the **"Access Level"** column > Facet > Text facet.
15. In the facet panel on the left, you will see every distinct value in that column with its count. Document all distinct values before writing GREL.
16. Repeat for Medicinal Uses and Ritual Use.
17. Compare the faceted values against the authorized controlled vocabulary list in Volume 1 Part 5. Any value that does not exactly match an authorized label must be corrected in the source spreadsheet before proceeding.

WARNING: Do not use GREL to silently correct misspellings or variant forms in controlled vocabulary columns. If a value does not match an authorized label, the error is in the data, not in the GREL. Fix the data in the source spreadsheet, re-export the CSV, and reload the project. GREL should map correct labels to slugs, not salvage incorrect data.

Step 3.2 — Split multi-valued cells

The following columns contain pipe-delimited multiple values and must be split into individual rows before configuring RDF output. Each pipe-delimited value will become its own triple in the output.

Columns to split:

- Plant Synonyms
- Yoruba Name(s)
- Name (Cuba)
- Name (Lucumi)
- Name (Haiti)
- Name (Vodou)
- Name (Brazil)
- Name (Candomblé)
- Other Vernacular Names

For each column in the list above:

18. Click the dropdown on the column > **Edit cells > Split multi-valued cells...**
19. In the separator field, enter the pipe character: |
20. Click OK.

NOTE: *The pipe delimiter in this dataset has no surrounding spaces: the separator is | not " | ". If your dataset was built following the Volume 1 no-space rule and the VBA utilities were applied correctly, the separator field should contain only the bare pipe character. If you see spaces around pipes in the facet values, the VBA cleanup was not completed. Close the project, fix the source spreadsheet, and reload.*

After splitting, the row count will increase substantially. This is expected — each multi-valued cell creates additional rows. All rows sharing a Plant ID represent the same plant record; the RDF Transform handles this correctly by grouping on the subject URI.

Verify the split worked correctly by spot-checking a plant record with known multiple values. Click on the Plant ID for that record and confirm the Yoruba names appear as separate rows rather than a single pipe-separated string.

Step 3.3 — Access Level standardization

The Access Level column contains human-readable labels from the controlled vocabulary. This GREL expression maps each label to its Iroko Framework concept slug. Apply it as follows:

21. Click the dropdown on **"Access Level"** > Edit cells > Transform...
22. Paste the expression below into the GREL expression field.
23. Click Preview to verify the mapping before clicking OK.

```
if(value == "Public - Unrestricted",      "access-public-unrestricted",
if(value == "Public - No Amplification",  "access-public-no-amplification",
if(value == "Limited - Community Only",   "access-community-only",
if(value == "Limited - Initiated Only",   "access-initiated-only",
if(value == "Private - Practitioner Access", "access-practitioner-access",
if(value == "Restricted - No Access",      "access-no-access",
value))))))
```

The final bare

`value` is the fallback — it returns the cell unchanged for any value not matched by the above conditions. After running this expression, apply a text facet to the Access Level column and confirm that all values now appear as slugs (starting with "access-") and that no human-readable labels remain. Any remaining labels indicate a value that did not match any condition — investigate the exact string in that cell.

Step 3.4 — Medicinal Uses standardization

24. Click the dropdown on **"Medicinal Uses"** > Edit cells > Transform...
25. Paste the expression below.
26. Click Preview to verify, then OK.

```
if(value == "Digestive support",          "medicinal-digestive-support",
if(value == "General tonic / revitalizing", "medicinal-general-tonic",
if(value == "Respiratory support",        "medicinal-respiratory-support",
if(value == "Skin and topical use",       "medicinal-skin-topical",
value))))
```

After running, facet the column and confirm all values start with "medicinal-".

Step 3.5 — Ritual Use standardization

27. Click the dropdown on **"Ritual Use"** > Edit cells > Transform...
28. Paste the expression below.
29. Click Preview to verify, then OK.

```
if(value == "Cosmological or Symbolic Association", "ritual-cosmological-symbolic",
if(value == "Healing and Restoration",              "ritual-healing-restoration",
if(value == "Invocation and Communication",          "ritual-invocation-communication",
if(value == "Offering and Devotion",                 "ritual-offering-devotion",
if(value == "Protection and Boundary Setting",       "ritual-protection-boundary",
```

```
if(value == "Purification and Cleansing", "ritual-purification-
cleansing",
if(value == "Rites of Transition", "ritual-rites-of-transition",
value))))))
```

After running, facet the column and confirm all values start with "ritual-".

Step 3.6 — GREL syntax notes for interns

GREL is not a general programming language — it is a lightweight expression language applied cell by cell. Three patterns cover everything needed in this workflow:

- `value` — returns the cell content unchanged. Used as the fallback at the end of if-chains.
- `value == "exact string"` — exact match test. Case-sensitive. Punctuation-sensitive. The string must match character for character.
- `"prefix" + value` — string concatenation. Used for URI construction in Part 4.

When a GREL expression is not mapping a value as expected, the most common causes are: (1) a trailing or leading space in the source cell that is invisible in the spreadsheet, (2) a different quotation mark character (curly quotes copied from Word), or (3) a capitalization difference. Use the text facet to see the exact string before writing the condition.

Part 4: URI Column Construction

Overview

RDF requires URIs for subjects (what each record is about) and for concept references (what controlled vocabulary terms mean). This part creates four new columns containing the URI forms of the Plant ID and three controlled vocabulary fields. These URI columns — not the original text columns — are what the RDF Transform skeleton uses for IRI-typed objects.

Step 4.1 — Create the Plant URI column

30. Click the dropdown on the **"Plant ID"** column > Edit column > Add column based on this column...
31. Name the new column **"Plant_URI"**
32. Paste the expression:

```
"https://ontology.irokosociety.org/data/ewe/" + value
```

33. Click OK.

This produces subject URIs in the form:

```
https://ontology.irokosociety.org/data/ewe/Plant0001
```

NOTE: The `/data/ewe/` path places records in the data namespace, separate from the vocabulary namespace at `/iroko#`. For production deployment these records should live at `data.irokosociety.org/ewe/Plant####` — a dedicated data subdomain — but this path is correct for current purposes. The subdomain migration is documented in *Known Limitations*.

Step 4.2 — Create URI columns for controlled vocabulary fields

For each of the three standardized concept columns, add a corresponding URI column. Use a null guard so that blank cells produce null rather than a malformed URI containing only the namespace prefix.

For Access Level:

34. Click the dropdown on **"Access Level"** > Edit column > Add column based on this column...
35. Name it **"Access_Level_URI"**
36. Paste:

```
if(isNotBlank(value), "https://ontology.irokosociety.org/iroko#" + value, null)
```

37. Repeat for Medicinal Uses — name the column **"Medicinal_Use_URI"**
38. Repeat for Ritual Use — name the column **"Ritual_Use_URI"**

These expressions produce URIs such as:

```
https://ontology.irokosociety.org/iroko#access-community-only  
https://ontology.irokosociety.org/iroko#medicinal-digestive-support  
https://ontology.irokosociety.org/iroko#ritual-healing-restoration
```

Step 4.3 — Verify URI columns

After creating all URI columns, apply a text facet to each one and confirm:

- No URI values end with the bare namespace (no cells containing only the prefix with nothing appended after #).
- Blank rows in the source column produced null in the URI column rather than a broken URI.
- The slug portion after # matches what you see in the Iroko browse pages exactly.

NOTE: *isNonBlank(value)* is equivalent to *(value != null && value != "")*. Either form works in GREL. *isNonBlank()* is preferred for readability.

Part 5: Taxonomic Reconciliation Using Global Names Verifier

What reconciliation does

Reconciliation links your dataset's free-text synonym strings to authoritative identifiers in external taxonomic databases. This confirms that a synonym is a recognized botanical name, identifies what accepted species it resolves to, and attaches a persistent external identifier to the record for downstream linked data use.

Step 5.1 — Which reconciliation service to use

The iPhylo reconciliation services for NCBI Taxonomy and GBIF that appear in older OpenRefine documentation are no longer functional in OpenRefine 3.9. The Wikidata reconciliation endpoint has shallow botanical coverage for West African species. Use the Global Names Verifier API:

```
https://verifier.globalnames.org/api/v1/reconcile
```

This service draws simultaneously from Catalogue of Life, World Flora Online, GBIF Backbone Taxonomy, WCVF, and other authoritative sources, and provides reliable coverage for the African and diasporic botanical synonyms in the Verger dataset.

Step 5.2 — Run the reconciliation

39. Click the dropdown on the **"Plant Synonyms"** column (after splitting multi-valued cells).
40. Select Reconcile > Start reconciling...
41. Click "Add Standard Service..." and enter the Global Names Verifier URL above.
42. Select the service from the list. Leave the type configuration on its default.
43. Click "Start Reconciling". OpenRefine queries Global Names for each unique value. This may take several minutes.
44. When complete, a reconciliation score appears in each cell. Values with strong matches will auto-match; values with lower confidence will offer candidate choices.
45. Review unmatched or uncertain cells. For well-documented botanical synonyms you should see high match rates.

Step 5.3 — Extract reconciled URIs

46. Click the dropdown on the reconciled **"Plant Synonyms"** column.

47. Select Edit column > Add column based on this column...

48. Name the new column **"Synonym_URI"**

49. Paste:

```
cell.recon.match.id
```

50. Click OK.

NOTE: Known limitation: Global Names Verifier returns internal UUID identifiers rather than fully qualified https:// URIs. In the exported Turtle these appear as relative URI references against the Base IRI. A production implementation would post-process these UUIDs by querying the Global Names API to retrieve the Catalogue of Life or GBIF species page URL for each matched record. This is documented in the Known Limitations section.

Part 6: Namespace Resolution Infrastructure

Why this section matters

The canonical Iroko namespace URI <https://ontology.irokosociety.org/iroko#> requires a document to exist at the /iroko path before the fragment (#) portion can resolve in a browser. Without this, clicking on an Iroko term URI in the exported Turtle returns a 404 even though the term is defined in the vocabulary. This is an infrastructure concern, not an OpenRefine concern — but it must be in place for the exported URIs to be fully dereferenceable.

There are two infrastructure fixes required. Both are documented here for completeness. For full deployment instructions, see Volume 4.

Fix 1: The [iroko.html](#) redirect file

A file named [iroko.html](#) must exist at the root of the [ontology.irokosociety.org](#) GitHub Pages repository. This file intercepts requests to the /iroko path and redirects them to the correct module HTML page based on the fragment identifier. The redirect works as follows:

- A browser requests <https://ontology.irokosociety.org/iroko#medicinal-respiratory-support>
- GitHub Pages serves [iroko.html](#) for the /iroko path
- JavaScript in [iroko.html](#) reads the fragment ([#medicinal-respiratory-support](#)) and looks it up in a 1,044-term mapping table
- The browser is redirected to the correct module page (e.g., [/vocab/iroko-ewe.html#concept-medicinal-respiratory-support](#))

This file exists in the repository. If you are verifying whether it is in place, check the repository root for `iroko.html`. If it is missing, see Volume 4 for the deployment procedure.

Fix 2: CSS anchor navigation

The HTML browse pages use a `.scheme-block` CSS class as a container for concept groups. This class originally had the declaration `overflow: hidden`, which creates a new scroll container in browsers and blocks anchor navigation — the browser can navigate to the page but cannot scroll to the specific concept element inside the container.

The fix is a one-character change in `iroko-style.css`:

```
/* BEFORE (incorrect) */
.scheme-block {
  overflow: hidden;
}

/* AFTER (correct) */
.scheme-block {
  overflow: clip;
}
```

`overflow: clip` preserves the visual clipping behavior needed for the border-radius styling but does not create a new scroll container. This allows browser anchor navigation to reach elements inside the container.

NOTE: This CSS change takes effect across all 17 HTML browse pages immediately on commit and push. Do not re-run the Python HTML generation scripts for a CSS-only change. Those scripts only need to run when TTL vocabulary files change (new terms, modified definitions). The CSS is presentational and independent of the vocabulary generation pipeline.

Part 7: RDF Transform Configuration

Overview

The RDF Transform extension maps your OpenRefine columns to RDF subject-predicate-object structures. This part describes how to configure the skeleton that controls what the exported Turtle file looks like. The skeleton is a template: it defines which column is the subject, which columns become predicates, and what type (IRI, literal, language-tagged literal, or boolean) each object should be.

Step 7.1 — Open the RDF Transform editor

51. In your OpenRefine project, click "RDF Transform" in the top menu bar.

52. Select "Edit RDF Transform..."

The RDF Transform dialog opens with a Transform tab and a Preview tab. Configure the skeleton in the Transform tab. Use the Preview tab to verify output before exporting.

Step 7.2 — Set the Base IRI

Near the top of the dialog, click "Edit" next to the Base IRI field and set it to:

```
https://ontology.irokosociety.org/
```

The trailing slash is required. The Base IRI is the namespace against which relative URIs in the skeleton are resolved.

Step 7.3 — Define namespaces

Click "Add" next to Available Namespaces and add the following four namespaces. For each, enter the prefix and the full IRI, then select "Add prefix only" — do not attempt to fetch vocabulary terms from the web, as this is unreliable for the Iroko namespace.

```
iroko    -> https://ontology.irokosociety.org/iroko#
dwc      -> http://rs.tdwg.org/dwc/terms/
skos     -> http://www.w3.org/2004/02/skos/core#
dcterms  -> http://purl.org/dc/terms/
```

The default namespaces (foaf, rdf, rdfs, owl, xsd, vcard) are pre-loaded and do not need to be added manually.

Step 7.4 — Configure the root node

The root node defines the subject of each set of triples. By default the extension assigns a row index. You want the subject to be the plant URI from the Plant_URI column.

53. Click on the root node (labeled "R: [Index]" by default).
54. In the RDF Node dialog, select the "Plant_URI" column from the content list.
55. On the right side, select "IRI" as the content type.
56. Click OK.
57. Click "**Add type...**" under the root node and enter `iroko:EwePlantRecord` to declare the class type for each record.

Step 7.5 — Complete predicate-object mapping

The following table defines the complete mapping for the Verger dataset v2 schema. For each column, configure the predicate name and object type as shown. Columns marked "IRI" in the Object Type column must use the URI column (e.g., `Access_Level_URI`), not the original text column (`Access Level`).

Column	Predicate	Object Type	Notes
Plant ID	dcterms:identifier	Text	Literal string ID
Plant_URI	dcterms:identifier	IRI	Subject URI — also set as root node
Scientific Name	dwc:scientificName	Text	
Plant Synonyms	skos:altLabel	Text	After splitting on pipe
Synonym_URI	dwc:taxonRemarks	IRI	Global Names reconciled identifier
Yoruba Name(s)	skos:altLabel	Language: yo	Standard Yoruba BCP 47 tag
Primary Vernacular Name	skos:prefLabel	Text	Quiros-Moran preferred label
Name (English)	skos:altLabel	Language: en	
Name (Cuba)	skos:altLabel	Language: es	Cuban Spanish — use @es not @sp
Name (Lucumi)	skos:altLabel	Language: x-lucumi	Private-use BCP 47 tag
Name (Haiti)	skos:altLabel	Language: ht	Haitian Creole BCP 47 tag
Name (Vodou)	iroko:vodouName	Text	No language tag — liturgical lang varies
Name (Brazil)	skos:altLabel	Language: pt-BR	Brazilian Portuguese
Name (Candomblé)	iroko:candombleName	Text	No language tag — liturgical lang varies
Other Vernacular Names	skos:altLabel	Text	Country-prefixed. No language tag.
Name Collision (Yes/No)	iroko:nameCollision	Boolean	
Collision Notes	iroko:collisionNote	Text	
Medicinal_Use_URI	iroko:medicinalUse	IRI	Use URI column, not source text
Access_Level_URI	iroko:accessLevel	IRI	Use URI column, not source text
Ritual_Use_URI	iroko:ritualUse	IRI	Use URI column, not source text
Ritual Notes	iroko:ritualNote	Text	

NOTE: The predicates *iroko:vodouName* and *iroko:candombleName* are Iroko Framework properties defined in the governance layer. Verify the exact property URIs against the Iroko browse pages at ontology.irokosociety.org before configuring the skeleton. If these

properties do not yet exist in the current framework version, consult the Executive Director — do not substitute a different predicate without authorization.

Step 7.6 — Critical rules for object type selection

Getting the object type right in the RDF Node dialog is the single most common source of errors in the skeleton configuration. Three rules:

- **URI columns must be set to IRI, not Text.** If an object node is set to Text, the full URI string outputs as a quoted literal ("https://...") rather than as a resource reference (iroko:access-no-access). This makes the triple invalid for linked data purposes.
- **Language-tagged literals do not need Text selected separately.** When you select Language and enter a language code (e.g., yo), OpenRefine automatically treats the value as a text literal with that language tag. The Language radio button and the Text radio button are mutually exclusive.
- **Boolean must be selected explicitly for Yes/No fields.** Text would output the strings "Yes" and "No". Boolean outputs xsd:boolean typed literals.
- **No language tag for Vodou and Candomblé columns.** These columns use plain Text object type. Do not apply a language tag. The liturgical language of these columns varies by record and cannot be generalized to a single BCP 47 tag.

Step 7.7 — Setting predicate names

By default, the RDF Transform skeleton uses the column name as the predicate, producing invalid predicates like :Access_Level_URI. To set the correct predicate:

58. In the Transform tab, click on the predicate label in the second column (the column name displayed there).
59. A text field opens. Clear it and type the correct property with its namespace prefix (e.g., iroko:accessLevel).
60. Press Enter or click away to confirm.

NOTE: *The predicate for URI columns should be the Iroko property name (iroko:accessLevel), not the column name (Access_Level_URI). The column name is a workflow label. The predicate name is what appears in the Turtle output.*

Step 7.8 — Save and use the Preview tab

Click Save at the bottom of the RDF Transform dialog frequently. The skeleton is not auto-saved. If the dialog is closed without saving, all configuration changes are lost.

After configuring each section, click the Preview tab to inspect the current Turtle output. Check the following before proceeding to export:

- Subject URIs appear as full `https://` URIs, not relative paths.
- Iroko concept references appear as CURIE notation (`iroko:access-community-only`) not as quoted strings.
- Language tags are correct: `@yo`, `@en`, `@es`, `@ht`, `@pt-BR`, `@x-lucumi`.
- Name (Vodou) and Name (Candomblé) values appear as plain text literals with no language tag.
- Name collision values appear as `"Yes"^^xsd:boolean` or `"No"^^xsd:boolean`, not as plain strings.

Part 8: Export and Post-Export Validation

Step 8.1 — Export as Turtle

61. Once the Preview looks correct, click Save in the RDF Transform dialog and then close it.
62. In the upper right corner of OpenRefine, click Export.
63. Hover over "RDF Transform".
64. Select either **Pretty Exports > Turtle** or **Stream Exports > Turtle (Stream)**.

The two export paths produce functionally identical Turtle output. Pretty Exports formats with indentation and blank lines between records (human-readable, preferred for review and submission). Stream Exports is more compact. Save the file with a descriptive name including the dataset name, schema version, and date (e.g., `Verger_Ewe_v2_2026-03.ttl`).

Step 8.2 — Post-export validation checklist

Open the `.ttl` file in a plain text editor and verify the following before considering the export complete:

Check	Pass	Fail
All prefix declarations appear at the top and reference correct namespaces	☐	☐
Subject URIs follow the pattern <code>https://ontology.irokosociety.org/data/ewe/Plant####</code>	☐	☐
<code>iroko:accessLevel</code> , <code>iroko:medicinalUse</code> , <code>iroko:ritualUse</code> appear as prefixed names not quoted strings	☐	☐
Language tags present: <code>@yo</code> , <code>@en</code> , <code>@es</code> , <code>@ht</code> , <code>@pt-BR</code> , <code>@x-lucumi</code>	☐	☐
Name (Vodou) and Name (Candomblé) values appear as plain literals with no language tag	☐	☐
<code>iroko:nameCollision</code> values appear with <code>^^xsd:boolean</code> datatype annotation	☐	☐
<code>iroko:collisionNote</code> values appear as plain text literals without datatype annotation	☐	☐

Check	Pass	Fail
iroko:vodouName and iroko:candombleName predicates are present for records with those values	☐	☐
No subject URI appears as a relative path	☐	☐
Record count in Turtle matches record count in source dataset	☐	☐

If any check fails, return to the RDF Transform skeleton, correct the relevant node configuration, and re-export. Save each export with a version suffix (v1, v2, etc.) so previous exports are retained for comparison.

Part 9: OpenRefine Project Preservation

Why this matters

An OpenRefine project contains more than your data. It contains the full transformation history — every GREL expression you applied, every cell split, every reconciliation result, and the complete RDF Transform skeleton configuration. If the project is lost, reproducing the exact same output requires rebuilding all of that from scratch using the documentation in this manual. That takes hours and introduces risk of inconsistency.

Preserving the project archive means future work can start from a known good state rather than starting from zero. It also means that when the Iroko Framework releases a new vocabulary version, the RDF skeleton can be updated against the existing project rather than reconfigured entirely.

Step 9.1 — Export the project archive

OpenRefine can export a complete project archive as a .tar.gz file that includes the data, the transformation history, and the RDF skeleton.

65. In the OpenRefine project, click Export in the upper right corner.
66. Select **"OpenRefine project archive to file"**.
67. Save the file. The default name includes the project name and a timestamp.

Rename the file using the standard naming convention:

```
[DatasetName]_[SchemaVersion]_[Date].openrefine.tar.gz

# Example:
Verger_Ewe_v2_2026-03.openrefine.tar.gz
```

Step 9.2 — What the archive contains

The .tar.gz archive contains:

- **data.zip** — the full dataset as it exists in the project at the time of export, including all transformation results, new columns, and reconciliation data.
- **history/ directory** — a record of every operation applied to the project in sequence. This is what allows OpenRefine to replay the transformation history.
- **metadata.json** — project metadata including name, creation date, and row count.
- **RDF Transform configuration** — the skeleton is embedded in the project data and is included in the archive.

NOTE: The archive does not contain the source CSV file. The source CSV should be preserved separately as the authoritative input record. Archive both the source CSV and the .openrefine.tar.gz file together in the same storage location.

Step 9.3 — Storage and naming conventions

Store project archives in the IHS data directory alongside the source CSV and the exported Turtle file. The three files that constitute a complete transformation record are:

```
[DatasetName]_[SchemaVersion]_[Date].csv           # source
[DatasetName]_[SchemaVersion]_[Date].openrefine.tar.gz # project
[DatasetName]_[SchemaVersion]_[Date].ttl           # output

# Example for Verger Ewé v2, March 2026:
Verger_Ewe_v2_2026-03.csv
Verger_Ewe_v2_2026-03.openrefine.tar.gz
Verger_Ewe_v2_2026-03.ttl
```

When a dataset is updated (new records added, existing records corrected), increment the date component rather than the schema version. Increment the schema version only when the column structure or controlled vocabulary mapping changes.

Step 9.4 — Restoring from a project archive

68. Open OpenRefine in your browser.
69. Click "Import Project" on the OpenRefine start page.
70. Select the .tar.gz archive file.
71. Click "Import Project".

The project will load with all data, transformation history, and RDF skeleton intact. The reconciliation results from Global Names are preserved in the archive — you do not need to re-run reconciliation unless the synonym data has changed.

NOTE: Project archives are version-locked to the OpenRefine version that created them. An archive created in OpenRefine 3.9 should be restored in OpenRefine 3.9. If OpenRefine must be upgraded, test the archive import on a copy before committing to the upgrade.

Step 9.5 — When to create a new archive versus update an existing one

Create a new archive (new filename with updated date) when:

- Any records have been added, removed, or modified in the source dataset.
- The RDF skeleton has been updated for a new framework version.
- Reconciliation has been re-run with updated results.
- Any GREL transformation has been added or changed.

Do not overwrite an existing archive. Every archive represents a specific reproducible state of the transformation. Overwriting it destroys the ability to reproduce that exact output.

Known Limitations and Future Improvements

Global Names Verifier UUID identifiers

The `dwc:taxonRemarks` property currently contains Global Names internal UUID identifiers rather than fully resolvable external URIs. This is a limitation of how the Global Names Verifier API returns reconciliation results. A production implementation would post-process these UUIDs by querying the Global Names API to retrieve the Catalogue of Life or GBIF species page URL for each matched record.

Data subdomain deployment

Plant record subject URIs currently use the path `https://ontology.irokosociety.org/data/ewe/Plant####`. For production, these should live at a dedicated data subdomain: `https://data.irokosociety.org/ewe/Plant####`. Vocabulary identifiers and data identifiers should not share a domain. The subdomain requires a separate CNAME configuration and GitHub Pages setup (see Volume 4).

Unused namespace prefixes

The exported Turtle declares foaf and vcard prefixes that are not used in the current Verger dataset output. These are harmless but could be removed in a post-processing step if the file will be published as a formal dataset release.

Spanish language tag

Earlier versions of this workflow used `@sp` as a language tag for Spanish, which is not a registered BCP 47 code. The correct registered tag is `@es`. The current manual specifies

@es throughout. If any existing Turtle files in the archive use @sp, they should be updated in the next export cycle.

Vodou and Candomblé predicate availability

The predicates iroko:vodouName and iroko:candombleName are specified in this manual as the correct predicates for the corresponding columns. These must be defined in the current version of the Iroko Framework before the exported Turtle will fully validate. If they are not yet defined, the Executive Director must authorize their addition to the framework vocabulary before a production export is made.

Appendix A: GREL Quick Reference

Controlled vocabulary standardization — Access Level

```
if(value == "Public - Unrestricted", "access-public-unrestricted",
if(value == "Public - No Amplification", "access-public-no-amplification",
if(value == "Limited - Community Only", "access-community-only",
if(value == "Limited - Initiated Only", "access-initiated-only",
if(value == "Private - Practitioner Access", "access-practitioner-access",
if(value == "Restricted - No Access", "access-no-access",
value))))))
```

Controlled vocabulary standardization — Medicinal Uses

```
if(value == "Digestive support", "medicinal-digestive-support",
if(value == "General tonic / revitalizing", "medicinal-general-tonic",
if(value == "Respiratory support", "medicinal-respiratory-support",
if(value == "Skin and topical use", "medicinal-skin-topical",
value))))
```

Controlled vocabulary standardization — Ritual Use

```
if(value == "Cosmological or Symbolic Association", "ritual-cosmological-
symbolic",
if(value == "Healing and Restoration", "ritual-healing-restoration",
if(value == "Invocation and Communication", "ritual-invocation-
communication",
if(value == "Offering and Devotion", "ritual-offering-devotion",
if(value == "Protection and Boundary Setting", "ritual-protection-boundary",
if(value == "Purification and Cleansing", "ritual-purification-
cleansing",
if(value == "Rites of Transition", "ritual-rites-of-transition",
value))))))
```

URI construction

```
# Plant subject URI from Plant ID column
"https://ontology.irokosociety.org/data/ewe/" + value

# Concept URI from slug column (with null guard for blank cells)
if(isNonBlank(value), "https://ontology.irokosociety.org/iroko#" + value, null)

# Extract reconciliation match URI
cell.recon.match.id

# Generic null guard pattern
if(isNonBlank(value), <your expression here>, null)

# Template for a new controlled vocabulary column
if(value == "Label A", "slug-a",
if(value == "Label B", "slug-b",
if(value == "Label C", "slug-c",
value)))
```

Appendix B: Transformation QC Checklist

This checklist covers the transformation phase. It is separate from the dataset QC Checklist in Volume 1 Appendix B, which must be completed first. Both checklists must be signed off before a Turtle file is considered ready for publication or submission.

Section 1: Pre-Transformation

Check	Pass	Fail
Volume 1 dataset QC checklist is signed off for this dataset version	☐	☐
Source CSV was exported as UTF-8 (not Windows-1252)	☐	☐
Diacriticals verified correct in a text editor before OpenRefine import	☐	☐
OpenRefine 3.9 is in use (not a newer or older version)	☐	☐
RDF Transform extension is installed and visible in the OpenRefine menu bar	☐	☐
Iroko Framework browse pages are accessible at ontology.irokosociety.org	☐	☐

Section 2: GREL and URI Construction

Check	Pass	Fail
Text facets confirmed on all three controlled vocabulary columns before applying GREL	☐	☐
Access Level: all values now start with "access-" (no human-readable labels remain)	☐	☐
Medicinal Uses: all values now start with "medicinal-" (no human-readable labels remain)	☐	☐
Ritual Use: all values now start with "ritual-" (no human-readable labels remain)	☐	☐
All nine pipe-delimited columns have been split on (no spaces around pipe)	☐	☐
Plant_URI column created and values follow correct URI pattern	☐	☐
Access_Level_URI, Medicinal_Use_URI, Ritual_Use_URI columns created with null guard	☐	☐
No URI column contains only the bare namespace prefix (null guard is working)	☐	☐
Reconciliation completed on Plant Synonyms using Global Names Verifier	☐	☐
Synonym_URI column created using cell.recon.match.id	☐	☐

Section 3: RDF Skeleton Configuration

Check	Pass	Fail
Base IRI set to https://ontology.irokosociety.org/ (with trailing slash)	☐	☐
All four namespaces defined: iroko, dwc, skos, dcterms	☐	☐
Root node configured to Plant_URI column with IRI object type	☐	☐
Root node has iroko:EwePlantRecord type declaration	☐	☐
All three URI columns (Access_Level_URI, Medicinal_Use_URI, Ritual_Use_URI) set to IRI — not Text	☐	☐
Name (Vodou) and Name (Candomblé) set to Text with no language tag	☐	☐
Name Collision column set to Boolean — not Text	☐	☐
Language tags correct: yo, en, es, ht, pt-BR, x-lucumi	☐	☐
All predicate names set correctly (not defaulting to column name)	☐	☐
Skeleton saved before export	☐	☐

Section 4: Export and Post-Export

Check	Pass	Fail
Turtle exported using Pretty Exports > Turtle	☐	☐
Output file named using standard convention: [Dataset]_[Version]_[Date].ttl	☐	☐
All prefix declarations present and correct in exported .ttl file	☐	☐
Subject URIs appear as full https:// URIs in output	☐	☐
Concept IRIs appear as CURIE notation (iroko:access-...), not quoted strings	☐	☐
All expected language tags present (@yo, @en, @es, @ht, @pt-BR, @x-lucumi)	☐	☐
iroko:nameCollision values have ^xsd:boolean datatype annotation	☐	☐
Name (Vodou) and Name (Candomblé) have no language tag in output	☐	☐
iroko:vodouName and iroko:candombleName predicates present for applicable records	☐	☐
Output record count matches source dataset record count	☐	☐
OpenRefine project archive exported and saved with matching filename	☐	☐
Source CSV, project archive, and Turtle file stored together in IHS data directory	☐	☐

Sign-Off

Dataset Version	Date	Executive Director
Notes:		

NOTE: File the completed checklist as:
QC_Transformation_[DatasetName]_[Version]_[Date].pdf alongside the three output files.